

Sabancı University

Faculty of Engineering and Natural Sciences

Computer Science and Engineering

PixelProof AI Image Detection and Reliability Evaluation

CS395 Internship Final Report

Efe Han Keleş

Student ID: 31994

Company: Türk Telekomünikasyon A.Ş.

Supervisor: Önder Çelebi

Supervisor title: Director

Internship: 20 July–18 September 2026

Approved duration: 40 internship days

Format: Hybrid

Submission date: 4 October 2026

Abstract

Türk Telekom provides telecommunications and digital services across Türkiye. During my CS395 internship, I developed PixelProof to investigate AI-generated image detection while reducing false alerts on real photographs. A second experimental module explored locating AI-edited regions. Development followed an iterative process: inspect errors, define the next experiment, measure the result and record the decision. Dataset records tracked acquisition and permitted use, while experiment and history logs retained both improvements and failed approaches. Early transfer failures and label errors shifted the focus toward data quality and reliable evaluation. E92's learned components used 12,269 training images represented under four processing conditions, producing 49,076 training views. A separate but repeatedly used development collection contained 160 real and 160 AI images. E92 detected 159 AI images in each of two evaluation conditions. Compared with E43, false alerts fell from 68 to zero on real originals and from 68 to 14 after resizing and JPEG compression. All 20 numerical checks passed, but one newly missed reference-detected AI image prevented full acceptance. The deliverables include trained research components, a local web demonstration and reproducible records. Model2 remains experimental. The project demonstrates improvement on a defined development collection, rather than universal detection or production readiness. Further work should use independent, diverse evaluation sources, keep calibration separate from evaluation and test localization with more editors.

Table of Contents

1. Introduction.....	4
2. Company information.....	4
2.1 Corporate profile and services.....	4
2.2 Organization and internship context.....	5
3. Project background.....	6
3.1 Department information.....	6
3.2 Initial project status.....	6
3.3 Motivation and problem definition.....	7
3.4 Related literature.....	7
4. Internship project.....	8
4.1 Project objective and scope.....	8
4.2 My responsibilities.....	8
4.3 Methodology and tools.....	9
4.4 Expected outcomes and deliverables.....	10
4.5 Project details.....	10
4.5.1 Early models and the first transfer failures.....	10
4.5.2 Data integrity and corrected conclusions.....	10
4.5.3 Data roles and processing conditions.....	11
4.5.4 E92 representation and learning.....	12
4.5.5 Development improvement and acceptance limits.....	13
4.5.6 Honest scores and the web demonstration.....	14
4.5.7 Broader checks and rejected improvements.....	15
4.5.8 Model2 localization and its limits.....	15
4.6 Results.....	18
5. Internship experience.....	18
5.1 Learning.....	18
5.2 Relation to undergraduate education.....	19
5.3 Major difficulties.....	19
5.4 A typical day.....	20
6. Conclusions.....	20
7. Recommendations.....	21
8. References.....	22
9. Appendices.....	24
9.1 Development criteria.....	24
9.2 Evidence and reproduction map.....	25

1. Introduction

The main purpose of PixelProof is to understand whether a photograph is AI generated or AI-manipulated. This project contains 2 modules. Module 1 simply investigates the whole image and classifies it based on the result. Module 2 investigates parts of the image and searches for AI editing. However, Module 2 remains experimental. One of the main goals during the development phase was to avoid incorrectly labeling real images as AI generated. Project results mostly improved on the development data, but classification results are not guaranteed to be reliable for every image. The main result is a web demo that allows users to upload images and try the system with a user-friendly UI.

My internship was between 20 July to 18 September 2026, with 40 approved work days. This report explains the research, development process, and experiment results. Experiments numbers such as E92 refers recorded experiment which is not separate independent test (Keleş, 2026) .Section 2: Company introduction. Section 3 explains the project background and the problem. Section 4 presents the development process and results. Section 5: Internship experience. The conclusions and recommendations are in Section 6 and Section 7. Section 8 contains the references, and Section 9 contains the appendices

2. Company information

2.1 Corporate profile and services

Türk Telekomünikasyon A.Ş. provides fixed, mobile, broadband, television and corporate data services to households, businesses and wholesale operators. Its history exceeds 185 years; its services were brought under one brand in 2016. It operates offices and communications infrastructure across 81 provinces, with 31,756 employees reported alongside June 2026 figures (Türk Telekom, 2026b).

Its registered address is Turgut Özal Bulvarı, 06103 Aydınlikevler, Ankara. My internship office was Fatih Sultan Mehmet Mah., Balkan Cad. No:49, 34771 Ümraniye, İstanbul. The website is <https://www.turktelekom.com.tr>; the supplied internship contact number is +90 312 555 93 92. Ownership is 60% Türkiye Wealth Fund, 25% Treasury and Finance Ministry and 15% publicly traded shares. Subsidiaries include TT Mobil, TTNET, Argela, İnnova, SEBİT and AssisTT (Türk Telekom, 2026a).

Turkcell was a relevant competitor discussed during the internship. Nokia supplies fixed and mobile network equipment and related services to the Türk Telekom group (Finnvera, 2025).

2.2 Organization and internship context

Figure 1 summarizes the published corporate structure: network and IT support infrastructure, commercial units serve customers, and support units cover finance, staff and coordination (Türk Telekom, 2026a, p. 33). My host unit was Planlama ve Geliştirme Müdürlüğü (Planning and Development Directorate), led by Önder Çelebi.

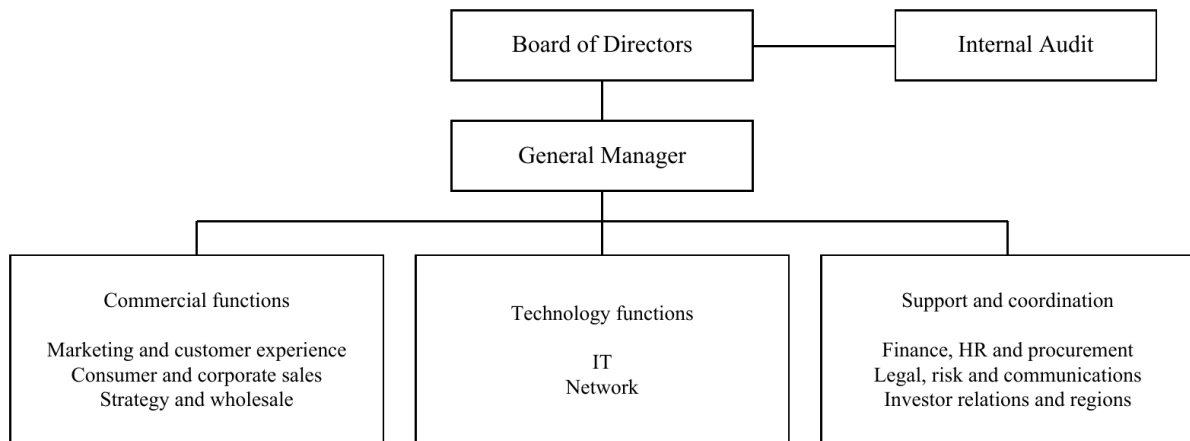


Figure 1. Simplified corporate reporting structure from the 2025 Annual Report.

3. Project background

3.1 Department information

My working unit was the Planning and Development Directorate, which focuses on Tivibu software and hardware. In this unit, my supervisor was Önder Çelebi (onder.celebi@turktelekom.com.tr), who is the director of this unit. My mentor was Hasan Çontuk (hasan.contuk@turktelekom.com.tr), who works in Önder Çelebi's team. This project was developed individually by me; there were no other group members throughout the internship. I asked my supervisor and mentor questions to help me develop the project properly. Office work days were Monday to Thursday, and the online day was Friday. This project was not developed for company use; it was just a student project. This project was not developed for company use; it was a student project.

3.2 Initial project status

The initial task was to prepare a dataset which includes real images and AI generated images to train the project's first model, a small CNN, and investigate classification errors. The dataset used was CIFAKE, and the model performed 96.75% accuracy on familiar test data. However, on a different dataset containing 995 images, accuracy decreased to 77.1%. After investigating this result, we found differences in image format and size between real and AI generated images. Therefore, we converted the images to a common JPEG format and cropped them into squares. When the test repeated there was no clear decrease in the performance of the small CNN. The workflow was to prepare the dataset, train the model, save the results, inspect errors and determine the next experiment (Keleş, 2026, E1 and E10).

3.3 Motivation and problem definition

The main problem of this project is misclassification, which has two main cases. The first one is a false positive: labeling a real photograph as AI generated. The second one is a false negative: failing to detect an AI generated image. For example, labeling more images as AI generated can help detect more AI generated images. However, it can also increase incorrect labels for real images. There are many factors to consider, such as different cameras, AI generators or image compression methods. When these factors differ between the training data and new images, they can affect the model's performance. This is called distribution shift, which means a change in the data distribution.

The main motivation of this project was to help users distinguish real photographs from AI generated photographs. The goal was to reduce false positive rates while maintaining the AI detection rate. Therefore, a web demo was developed to allow users to upload an image and view the model's result through a simple interface. The demo also shows an uncertain result when the system cannot make a clear decision.

3.4 Related literature

Wang et al. (2020) investigated detection across different image generators. Ojha et al. (2023) showed that pretrained visual features can support broader detection through simple classifiers. CLIP learns from image–text pairs (Radford et al., 2021), while DINOv2 learns reusable features without task-specific labels (Oquab et al., 2024). These works motivated feature-based transfer learning in PixelProof.

Compression and image-size differences can become misleading dataset shortcuts (Grommelt et al., 2024). DEAR prunes features using inpainting-based analysis to improve robustness (Kim et al., 2026). PixelProof used a pretrained DEAR component, but its published benchmark results do not establish our system's performance.

SIDD provides smartphone images from ten scenes and five cameras (Abdelhamed et al., 2018); SID provides short- and long-exposure low-light raw pairs (Chen et al., 2018). These sources supported real-image coverage, with limited scene diversity and processing conditions that differ from ordinary camera JPEGs.

Guo et al. (2017) explain why classifier scores need calibration before interpretation as confidence. Dwork et al. (2015) show the risks of repeatedly adapting to holdout results. Both issues shaped our evaluation and interface. As an industry comparison, Adobe Content Credentials records origin and editing information (Adobe, 2026). PixelProof estimates evidence from pixels; credential verification was not implemented.

The literature suggested useful starting points, but it could not settle the project question. A method that performs well in its own paper may use different generators, cameras or decision rules. I therefore used these studies to choose experiments and explain possible failures. The report separates published findings from measurements made in PixelProof, so that the reputation of a pretrained model does not stand in for local evidence.

4. Internship project

4.1 Project objective and scope

Model1 aimed to reduce real-photo false alerts while preserving AI detections already made by a reference model. Model2 explored edit localization. The scope covered data auditing, learned adaptation, evaluation and a local web demo with uncertainty handling. Universal detection and production readiness were longer-term goals, not achieved outcomes.

4.2 My responsibilities

I directed the experiments, organized data, reviewed results and generally made project decisions, consulting my supervisor and mentors for advice. They explained concepts such as true positives and false negatives. My work covered dataset checks, model adaptation, the

API, interface and research records. AI coding assistance supported implementation, analysis and documentation; scripts and tests checked outputs. Pretrained models and external datasets are credited separately from our adaptation and application work.

4.3 Methodology and tools

In later experiments, PLAN.md recorded the question and acceptance rules before a run. ml/EXPERIMENTS.md retained measurements, HISTORY.md explained decisions and DATASETS.md recorded sources and permitted use. Corrections remained alongside earlier entries. Table 1 shows how findings changed the next step (Keleş, 2026).

Table 1. Examples of findings that changed development.

Finding	Response
Reversed source labels	Remap labels and rerun experiments
Evaluation-dependent threshold	Use calibration data only
Better average but new AI misses	Reject the candidate

Training fitted the model; calibration selected thresholds; development comparisons guided changes. Final evaluation required data uninvolved in those decisions. I tracked AI recall, real-image false alerts and AUC, which measures ranking across thresholds. Python, PyTorch, NumPy, SciPy and scikit-learn supported modelling; React, TypeScript and FastAPI supported the demo. Git, saved scores, artifact hashes and automated checks made changes traceable.

For each candidate, the practical question was whether the change solved a known problem without creating an unacceptable one elsewhere. A lower total error count was insufficient if the model newly missed an AI image that the reference had detected. Keeping the failed candidates in the records helped explain why a newer experiment did not automatically replace the current demo model.

4.4 Expected outcomes and deliverables

The deliverables were trained research components, a local upload demo, reproducible experiment scripts and documented data and results. Model2 contributed an experimental localization evaluator. The prototype was intended for demonstration and further study.

4.5 Project details

4.5.1 Early models and the first transfer failures

The first CNN used 90,000 CIFAKE training images, 10,000 validation images and a 20,000-image familiar test. A pretrained ResNet-18 reached 97.66% on the familiar test but only 25.2% on the separate 995-image collection. Matching the training resolution recovered much of this loss. A stronger architecture alone had not solved source transfer (Keleş, 2026, E1–E6).

Native-resolution statistics, tile features, external detectors and combined scores were then compared. Gains on particular sources often came with new real-photo errors or missed AI types. Fixed crops and camera-noise explanations also needed controls rather than being assumed reliable. These failures shifted attention toward data quality, source coverage and explicit acceptance rules (Keleş, 2026, E8–E50).

I treated the familiar test as a check that the training procedure worked on similar data. The external collection asked a harder question about transfer to other sources. The large gap between them showed why one accuracy number was not enough. It also motivated checking input preparation before making a more expensive model change.

4.5.2 Data integrity and corrected conclusions

Two datasets used numeric labels opposite to our convention of 0 for real and 1 for AI. Explicit mapping and reruns corrected affected experiments. DINOv2's Defactify AUC

changed from 0.480 to 0.764, invalidating an earlier explanation that its features were unsuitable. The larger training pool also reduced false alerts more than first reported. A separate threshold-selection procedure was repaired because it could consult evaluation data; the corrected candidate failed admission and was removed (Keleş, 2026, E19b–E19c and E27).

A label is the answer supplied during learning or evaluation. Reversing its meaning can make a useful model appear poor and can teach a new model the wrong relationship. The repair therefore required more than changing a chart caption. Affected measurements had to be rerun, and the earlier explanation had to be corrected in the written record.

4.5.3 Data roles and processing conditions

A parent is one image before project-created transformations. E92 training used 12,269 parents: 7,674 real and 4,595 AI. Four processing conditions produced 49,076 views. These comprised clean input, an assigned transport transformation, JPEG75 and a social-style view capped at 1080 pixels before JPEG75 encoding. Downloaded archive size, selected training data and evaluation size are different quantities. Source, licence, duplicate and known ancestry checks governed admission.

Table 2 separates the major populations; its rows are not additive independent evidence. Earlier E42 and E43 evaluations failed acceptance. E43 passed 11/20 checks on 1,000 real plus 1,000 AI images, with 391 original and 490 processed real false alerts. E92 never reached that final stage because its preceding acceptance rule failed (Keleş, 2026, E42, E49, E92 and E139).

Table 2. Training and evaluation populations at different stages.

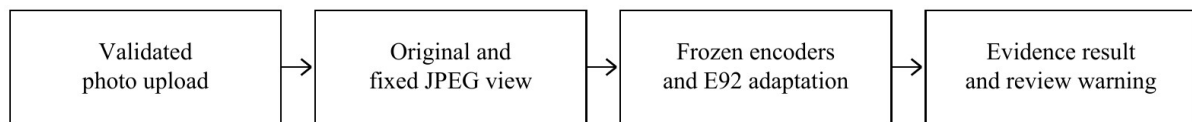
Population	Parents	Views	Role
E42 evaluation	16,953	50,858	Earlier model evaluation
E43 on E49-C	2,000	4,000	Earlier final evaluation
E92 training	12,269	49,076	Training
E92 on E66	320	640	Reused development
Later research	12,525	50,100	Internal diagnostics

The same photograph can appear at several sizes or compression levels. These versions help reveal sensitivity to processing, but they still come from one parent. Similarly, different photographs of the same scene may share useful clues. Counting views, files or downloaded gigabytes as independent examples would overstate the strength of the evidence. The useful question is which sources, scenes and generators a measurement actually covers.

4.5.4 E92 representation and learning

Fine-tuning, grayscale inputs and early score corrections failed the full requirements. Later work added CLIP and DEAR features and learned adaptation; E83 and E86 passed 17/20 development checks before E92 reached 20/20. This sequence alone does not isolate which addition caused improvement (Keleş, 2026, E51–E92).

E92 uses feature-based transfer learning. Pretrained DINOv2, CLIP and DEAR components remain fixed, while project-trained components learn to use their features. A constrained correction adjusts the reference score to reduce real-image errors while protecting AI responses on training examples. That protection still needs evaluation on other images. Figure 2 shows how the learned system connects to the interface.



Raw model scores remain separate from authenticity claims.

Figure 2. PixelProof inference flow from upload to displayed evidence.

Transfer learning reuses knowledge learned before this project. Here, an encoder converts pixels into a numerical description of the image. Keeping the encoder fixed means its pretrained weights stay unchanged while smaller project components learn from those descriptions. This made adaptation practical on the available computer. It also means that our contribution is the adaptation, evaluation and application around the existing representations, rather than the creation of DINOv2, CLIP or DEAR.

4.5.5 Development improvement and acceptance limits

Figure 3 compares the same 160 real development images. False alerts fell from 68 to zero for originals and from 68 to 14 for processed views. E92 detected 159 of 160 AI images in each condition. Ten numerical requirements per condition passed, but one original AI image previously detected by E43 became a new miss. The separate retention rule therefore failed.

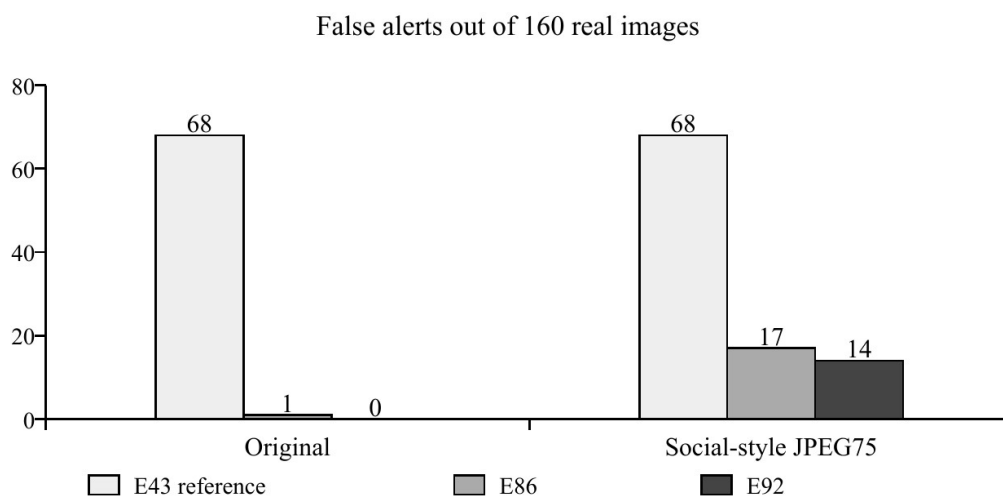


Figure 3. False alerts among 160 real development images per condition.

This reused collection contains only ten real SIDD scenes and two previously seen AI families. Thirteen of the fourteen processed real errors cluster in two scenes. Repeated use influenced later decisions, limiting generalization claims. E102 lowered processed false alerts to 12/160 but retained the same reference-relative AI miss, so it was not promoted (Keleş, 2026, E92 and E102).

The comparison asks both how many errors occurred and which images caused them. Two models can detect the same number of AI images while missing different examples. Under the retention rule, recovering one example cannot compensate for newly missing another. This explains how E92 could improve the overall measurements and still fail the full decision rule. It also keeps the report from treating 20/20 as proof of perfect classification.

4.5.6 Honest scores and the web demonstration

The demo's 0–100 scale is a model score, not an AI probability. Its upper threshold is about 7.94, inherited from E48 calibration; its lower threshold is about 1.15, from already-used E49 calibration/development analysis. An original score at or above the upper threshold retains an AI alert. Two E92 scores below the lower threshold yield no clear AI evidence; other cases are uncertain. E43 provides advice without vetoing two low E92 scores. Decisions use unrounded values.

On the same development collection, the combined rule gave 139 real images no clear AI evidence and left 21 uncertain, with zero real AI alerts. It alerted on 159 AI images and left one uncertain. Thus per-view false alerts and final website warnings are different measurements. Upload validation, artifact checks and cancellation handling improve software operation, not measured detection accuracy (Keleş, 2026, current-policy audit).

The second processed view acts as a consistency check. When ordinary resizing and compression change the response, the user needs to know that the evidence is unstable. The interface therefore has an uncertain outcome instead of forcing every upload into a real-or-AI

answer. Even the no-clear-evidence result only describes what the detector found. Establishing where a photograph came from requires additional information about its source.

4.5.7 Broader checks and rejected improvements

The owner gallery produced 9 false alerts among 206 real originals and 18 after processing. Mostly one phone and related scenes, it had already influenced development and was not independent. Separate source-fold research models also failed full acceptance; their results do not measure active E92 performance (Keleş, 2026, E95 and E146).

A processing audit found known 224-pixel derivatives in 29.18% of real training images versus 7.68% of AI images. Added source-pixel features slightly improved clean averages but harmed processed AI detection; one JPEG75 result fell from 75.73% to 74.15%. Follow-up decomposition identified harmful score contributions, without proving a physical cause. The candidate was rejected. Unscored reserves remained, but no balanced, independent final test was established (Keleş, 2026, E148–E152).

These checks changed the next development priority. Adding another feature was less useful than learning whether the apparent improvement survived a change in source or processing. A suitable final evaluation must therefore be planned before inspecting its results. Its images should not have influenced training, threshold selection or decisions about which candidate to keep. More files from the same familiar sources would not resolve this problem by themselves.

4.5.8 Model2 localization and its limits

Model2 predicts edited regions and checks authentic images for false alarms. An early overlap evaluation improperly used true mask size to choose predicted area. A corrected test on 120 CocoGlide images performed worse than a central-region baseline (Keleş, 2026, E17 and E107).

Finding data was not the same as finding suitable training evidence. Of 512 CocoGlide originals, 506 shared source identities with protected project data, preventing new training admission under our split policy. The 512 downloaded DiffSeg30k pairs included 201 partial, 224 full-image and 87 empty masks; original-image ancestry and mask interpretation remained unresolved. They were kept out of training. Thus the project lacked a broad, validated local-edit dataset, despite having image files and masks (Keleş, 2026, E105, E109 and E117).

A local editing request also did not guarantee an unchanged background. In our Stable Diffusion 1.5 pilot, the 11 accepted outputs from 16 attempts changed an average of 99.94% of pixels outside the intended mask. Mean absolute channel change was about 9.79 on a 0–255 scale. Pixel differences do not mean that the scene changed semantically, but the raw background could not be labelled untouched. Explicit composites restored original pixels outside the mask; their boundaries still needed controls (Keleş, 2026, E122 and E127).

Revised input preparation later passed all 16 generation checks. We used these controlled composites, authentic images and conventional-edit controls for a small learning pilot. The masks defined the pasted region, not a verified semantic-change boundary. One editor and sixteen reused parents could not establish broad reliability (Keleş, 2026, E129–E135).

Table 3 shows that training on two edit placements improved the new placement but harmed the original placement and increased false markings on authentic images. The candidate was rejected. Moving edits also changes content, so the result does not isolate location as the sole cause. Model2 remains experimental.

Table 3. Model2 trade-offs on sixteen reused parents, original-image condition.

Endpoint	Earlier head	Two-placement head
Original-placement pixel AUC	0.74	0.70
New-placement pixel AUC	0.57	0.64
Authentic falsely flagged area	17.11%	25.74%

A localization map gives a score at each location. To assess it, we need a mask identifying the edited pixels and authentic images that should not contain an edited region. Pixel AUC measures how well scores rank edited pixels above authentic ones across thresholds. It is not the percentage of an edit correctly outlined. False markings on authentic images provide a separate practical check.

4.6 Results

The completed outcome is an E92 research system, local demo and reproducible development record. Table 4 summarizes per-view results on 160 real plus 160 AI development images, not the paired website outcomes in Section 4.5.6.

Table 4. E92 per-view results on the reused development collection.

Measure	Original	Social-style
AI detection	159/160 (99.38%)	159/160 (99.38%)
Real false alerts	0/160 (0.00%)	14/160 (8.75%)
Balanced accuracy	99.69%	95.31%
Numerical criteria	10/10	10/10
New AI misses versus E43	1	0

The numerical improvement did not meet full acceptance or prove universal reliability. Model2 also remained experimental. The engineering checkpoint passed 1,245 Python tests, which check software behavior rather than image accuracy. No production rollout or company deployment impact was measured (Keleş, 2026).

5. Internship experience

5.1 Learning

There are several things that I learned from this project. One of them is learning how to manage a long term and complex project individually. When the project became too complex to understand, I learned how to narrow down the scope and find a clearer way to ask questions. Another lesson is that good scores in a specific case do not prove success in general. For example, the first model scores were promising, but the model did not perform as well on other datasets. I also carried out practical experiments, learned new machine

learning concepts and understood concepts I already knew better. All of these strengthened my interest in the field.

The records helped me manage the work. After each experiment, I recorded what changed, what the result meant and what the next step should be. This made the process easier to understand and helped me avoid to apply approaches that I already rejected in past experiments.

5.2 Relation to undergraduate education

CS 412 Machine Learning course has a huge impact of my project development since this course taught me the basic concepts in Machine Learning, for example my first experiments has a CNN model. In Proj 201 course, our team trained a model called YOLOv11, this course introduced me model training and our team placed first among all groups. DSA 210 help me to understand how to find data, process it, understand it and evaluate the results.

5.3 Major difficulties

There are several difficulties that I encountered. The first one is the model's performance decreasing on other datasets. The model performed well on familiar datasets, but its performance decreased on other datasets. This is related to OOD (out of distribution) data. For example, if the model is trained on images with a fixed size, format and compression level, its performance might decrease when it encounters different types of images. To address this problem, I increased the variety of datasets, inspected which data the model made mistakes on and developed methods to improve its performance. However, reliable performance is not guaranteed on every new dataset.

The second one is ensuring that experiment results are measured properly. For example, during development, some datasets used different labels for real and AI images. Our code interpreted these labels in reverse, so the measured performance dropped. Noticing this

problem required careful investigation of the results. After correcting the label mapping, I repeated the affected experiments.

The third and last one is hardware and time limitations. Processing large numbers of images and large images might take time and strain our system. For example, during one run, a memory error interrupted the process. To prevent this, I developed procedures such as processing images in groups, limiting hardware usage and saving numerical features that the model extracts. Therefore, there is no need to calculate these features again for other experiments that use them.

5.4 A typical day

I would get to the office at 7:30 a.m. usually. I worked until after breakfast and until the break for lunch. Before lunch I would usually take one or two breaks for personal needs or tea and coffee. After lunch I worked on until 4:00 p.m. and left the office. This routine applied from Monday to Thursday, since Friday was a remote working day. A typical remote day followed a similar routine, but at home.

6. Conclusions

This project became a working system with a demo that allows users to upload images. The project benefited from unsuccessful experiments because they helped guide new approaches and methods. Model performance improved on the development data. However, high AI detection rates or low false positive rates do not guarantee reliable results for every image. These improvements were observed in Model 1. For Model 2, it is too early to make similar claims because it needs more experiments and suitable data. Model 2 remains experimental.

7. Recommendations

Students should meet with their supervisor and mentor before the internship begins to introduce themselves and to describe their interests and the field they would like to work in. This may help set expectations and help them find appropriate tasks, even if the work that is available is not always of interest. Students should also learn the basic concepts and tools related to their assigned work.

Another recommendation is to communicate clearly with colleagues, supervisors and mentors. Students are there to gain work experience and learn new skills, so they should ask questions when they need to, and be amenable to feedback. They can benefit more from the internship by sharing progress regularly and keeping notes about tasks done after working hours.

8. References

- Abdelhamed, A., Lin, S., & Brown, M. S. (2018). A high-quality denoising dataset for smartphone cameras. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 1692–1700. Retrieved September 29, 2026, from <https://abdokamel.github.io/sidd/>
- Adobe. (2026, August 25). Content Credentials overview. Adobe Help Center. Retrieved September 29, 2026, from <https://helpx.adobe.com/creative-cloud/apps/adobe-content-authenticity/content-credentials/overview.html>
- Chen, C., Chen, Q., Xu, J., & Koltun, V. (2018). Learning to see in the dark. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 3291–3300. Retrieved September 29, 2026, from <https://cchen156.github.io/SID.html>
- Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reingold, O., & Roth, A. (2015). Generalization in adaptive data analysis and holdout reuse. *Advances in Neural Information Processing Systems*, 28. Retrieved September 25, 2026, from <https://proceedings.neurips.cc/paper/2015/hash/bad5f33780c42f2588878a9d07405083-Abstract.html>
- Finnvera. (2025, October 23). Finnvera guarantees Nokia’s deliveries to Türk Telekom. Retrieved September 29, 2026, from <https://www.finnvera.fi/eng/export-and-internationalisation/finnvera-as-an-export-finance-provider/guaranteed-transactions/finnvera-guarantees-nokias-deliveries-to-turk-telekom>
- Grommelt, P., Weiss, L., Pfreundt, F.-J., & Keuper, J. (2024). Fake or JPEG? Revealing common biases in generated image detection datasets. *arXiv:2403.17608*. Retrieved September 25, 2026, from <https://arxiv.org/abs/2403.17608>
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of Machine Learning Research*, 70, 1321–1330. Retrieved September 25, 2026, from <https://proceedings.mlr.press/v70/guo17a.html>
- Keleş, E. H. (2026). PixelProof experiment, history and dataset records [Software and research records, commit 497d45c]. GitHub. Retrieved September 25, 2026, from <https://github.com/EfeHanKeles346/ai-image-detector/tree/497d45c>
- Kim, D., Choi, J., Seong, H. S., Kim, S., Lee, D., Yi, S., & Choi, J.-H. (2026). Dissect and prune: Enhancing robustness in AI-generated image detection. *ICML 2026*; *arXiv:2606.10309*, version 2. Retrieved September 25, 2026, from <https://arxiv.org/abs/2606.10309v2>
- Ojha, U., Li, Y., & Lee, Y. J. (2023). Towards universal fake image detectors that generalize across generative models. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 24480–24489. Retrieved September 25, 2026, from <https://arxiv.org/abs/2302.10174>

- Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., Synnaeve, G., Xu, H., Jégou, H., Mairal, J., Labatut, P., Joulin, A., & Bojanowski, P. (2024). DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*. Retrieved September 25, 2026, from <https://arxiv.org/abs/2304.07193>
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. *Proceedings of Machine Learning Research*, 139, 8748–8763. Retrieved September 25, 2026, from <https://proceedings.mlr.press/v139/radford21a.html>
- Türk Telekom. (2026a). 2025 annual report. Türk Telekom Investor Relations. Retrieved September 25, 2026, from <https://www.ttyatirimciiliskileri.com.tr/media/dqsjlvxo/2025-annual-report.pdf>
- Türk Telekom. (2026b). Hakkımızda [About us; operating figures at June 30, 2026]. Retrieved September 25, 2026, from <https://www.turktelekom.com.tr/hakkimizda/detay?p=tab3>
- Wang, S.-Y., Wang, O., Zhang, R., Owens, A., & Efros, A. A. (2020). CNN-generated images are surprisingly easy to spot... for now. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8695–8704. Retrieved September 25, 2026, from <https://arxiv.org/abs/1912.11035>

9. Appendices

9.1 Development criteria

Table 5 shows the 10 numerical conditions for the model and the results of E92. These conditions were evaluated separately for original images and images processed to simulate social media sharing. Therefore, there are 20 checks in total. These conditions were defined for this project. They are not a universal success standard or a certification. Automatic coverage means the proportion of images given a decision rather than an uncertain result. Coverage and uncertainty add up to 100%, and some other metrics are also related. Therefore, the 20 checks are not independent measures of success. Although E92 passed these numerical checks, it missed an AI image previously detected by the reference model and failed the separate requirement to preserve those detections. The values in Table 5 describe each image version separately, not the combined website results explained in Section 4.5.6.

Table 5. E92 numerical development checks and their measured values.

Criterion	Requirement	Original	Social
Valid-score coverage	100%	100%	100%
ROC-AUC	≥ 0.90	0.9998	0.9942
Balanced accuracy	$\geq 85\%$	99.69%	95.31%
Pooled real false alerts	$\leq 10\%$	0.00%	8.75%
Worst real source	$\leq 20\%$	0.00%	18.75%
Pooled AI detection	$\geq 80\%$	99.38%	99.38%
Worst AI source	$\geq 60\%$	98.75%	98.75%
Automatic coverage	$\geq 80\%$	98.75%	98.44%
Accuracy when automatic	$\geq 95\%$	99.68%	95.24%
Uncertain fraction	$\leq 20\%$	1.25%	1.56%

9.2 Evidence and reproduction map

Full experiment settings, corrections and aggregate evidence remain in the repository snapshot cited as Keleş (2026). Private images and large weights are excluded from Git.